1. Programare dinamică

Pentru a rezolva o problemă de programare dinamică, trebuie să stabilim dinamica problemei cât și dimensiunea ei. Apoi trebuie rezolvată problema prin recurență între soluția curentă ce vrem să o determinăm și soluțiile anterioare deja cunoscut. Rezultatul e ultimul element al s dim structura de date

Greedy vs DP

- sunt menținute soluții parțiale din care evoluează stările parțiale următoare
- soluțiile parțiale sunt eliminate
- necesită un criteriu optim
- conduce la optim local
- algoritm top-down

- se păstrează soluții parțiale
- la construcția unei soluții noi, orice soluție parțială construită anterior poate contribui
- nu necesită criteriu optim
- algoritm bottom-up

Problema cu paranteze

$\forall 1 \le i \le j \le n$

- $A_{ij} = A_i \cdot A \ldots \cdot A_j$. A_{ij} are p_i linii și p_j coloane.
- $m[i,j]$ = nr. optim de înmulțiri pt a rezolva subproblema $A_{i,j}$
- $s[i,j]$ = poziția primei paranteze pt A_{ij}

problema inițială: A_{1n}

Scanned with CamScanner

Combinarea subproblemelor

- Pt $A_{i,j}$, trebuie găsit indicele $i \le k \le j$ care asigură parantezare optimă

$$A_{i,j} = (A_i \times .. A_k) \times (A_{k+1} \cdot \dots \cdot A_j)$$

$$A_{i,j} = A_{ik} \times A_{k+1,j}$$

Algoritm optimal: căutăm optimul dintre toate variantele posibile.

ex. $n = 4$, $A(2,3)$, $B(3,4)$, $C(4,2)$, $D(2,3)$

i / j	0	1	2	3	4
0	2	3	4	2	3

- $((AB)C)D) = 24 + 16 + 12 = 52.$

  $X = (AB) \to 2*3*4 = 24$

  $Y = XC \to 2*4*2 = 16$

  $Z = YD \to 2*2*3 = 12.$

- $(A(BC)D = 24+12+12 = 48$

  $X = (BC) \Rightarrow 3*4*2 = 24$

  $Y = (A\,X) \to 2*3*2 = 12$

  $Z = YD \to 2*2*3 = 12.$

- alte parantezări: $(AB)(CD)$

  $A((BC)D)$

  $A(B(CD))$

Scanned with CamScanner

Problema Rucsacului

Recurență la dp: $dp[0][cap] = 0,\ cap = 0 : W$

$dp[i][cap] = max(dp[i-1],$

$dp[i-1][cap - w_i] + p_i)$

$i = 1, n$

Greedy | DP

Date de intrare: (w_i - greutate obiect, p_i - prețul obiectului)

se dă un rucsac cu volum infinit dar care suportă o greutate maximă W

Obiectele pot fi tăiate (varianta continuă)	Obiectele nu pot fi tăiate (varianta discretă a problemei)
se pot selecta fracții de obiecte	obiectele trebuie să fie întregi și selectate o singură dată
$T = \Theta(n)$ $W = \Theta(n)$	$T = O(n * W)$ $S = O(n * W$

Scanned with CamScanner

2. Numărul maxim și minim de CTC-uri într-un graf cu n noduri

max: n (doar noduri izolate)

minim: 1 (ciclu)

dacă avem top-sort : nu putem avea ciclu

3. Algoritmul pompare preflux – Push-relabel

- simularea curgerii lichidelor într-un sistem de conducte ce leagă noduri aflate la diverse înălțimi
- există un preflux inițial în rețea obținută prin încărcare la capacitate maximă a tuturor conductelor care pleacă din sursă
- când un nod u are flux disponibil în rezervor și o conductă spre un alt nod v nu e încărcată complet → înălțimea lui e crescută pentru a permite curgerea către v
- înălțime maximă: $2(|V|-1)$
- nr. total de înălțări: $2|V|^2$
- pompări – saturate: $2|V||E|$
  nesaturate: $4|V|^2(|V|+|E|)$
- complexitate totală: $O(V^2 \times E)$

Scanned with CamScanner

4. Backtracking

Funcția REVISE

- complexitate $O(a^2)$
- apelată pentru un arc al grafului de restricții binare și șterge acele valori din domeniul de definiție al unei variabile pentru care nu e satisfăcută restricția pentru nici o valoare corespunzătoare celeilalte variabile a restricției

AC1	AC3
se aplică algoritmul de consistență nodurilor, apoi se aplică REVISE până nu se mai realizează nicio schimbare $O(na * 2n * a^2)$	se elimină pe rând arcele se fac mai puține apeluri Revise Incon: reverificarea nodurilor direct implicată de o constrângere din rețea dacă o constrângere aduce modificări în rețea, adaugăm în travers. noduri care punctează către nodul de plecare al restricției verificate $O(a^3n^2)$

5. Algoritmul Johnson

- returnează o matrice ce conține costurile drumurilor minime pentru toate perechile de vârfuri, resp. False dacă 3 cicluri de cost negativ
- adaugă un nou vârf, unit prin toate celelalte cu arce 5

Scanned with CamScanner

Relaxări minime și maxime:

- Bellman-Ford: - numărul maxim coincide cu cel minim, deoarece sunt făcute atâtea iterații câte noduri și muchii avem.

$$-(V-1)\cdot E$$

- Dijkstra: - pot fi produse maxim V iterații, câte una pentru fiecare nod în parte (V-1 dacă nu includem nodul de start)
  - numărul minim nu cred că e standard, decât dacă știm structura grafului

7. Rețele de transport și flux maxim

- Transformarea unei probleme de determinare a fluxului maxim într-o rețea cu mai multe surse și mai multe destinații ⟹ problemă cu o singură sursă și destinație:
  - se construiește o sursă nouă
    - o legăm cu INF de toate sursele inițiale și o destinație nouă
  - legăm și destinația cu INF la toate destinațiile inițiale

- Graf rezidual:
  - graf care conține doar muchii nesaturate
  - cale reziduală = drum care pleacă din sursa de la flux și merge până în destinație doar pe muchii nesaturate;

Scanned with CamScanner

- Capacitatea reziduală = cantitatea cu care se poate mări fluxul pe arcul (u,v)
- Tăieturi în rețele de flux = partiționare a nodurilor în 2 mulțimi disjuncte
- Tăietură minimă : - flux maxim
  - rețeaua reziduală G_f nu are căi reziduale
  - există o tăietură (S,T) a.î. $|f| = c(S,T)$
  - f e o funcție de flux G a.î. $|f|$ e flux maxim total în G
- Algoritmul Fold-Fulkerson: (îmbunătățiri)
  - se alg căile reziduale cu capacitate maximă (Complexitatea depinde de f_{max} și valoarea capacităților)
  - se alg căile reziduale cele mai scurte (Complexitatea depinde de nr. de arce)
  - complex = $O(E * f_{max})$
- Edmons Karp:
  - $O(E^2 \cdot V)$
  - rezolvă problema drumurilor de ameliorare folosind BFS
  - implementare concretă a alg. Fold - Fulkerson

8. Algoritmi euristici

- dacă $h(u) = 0$ la A^* $\Rightarrow$ Dijkstra

Scanned with CamScanner

- 8 puzzle: trebuie aranjat pornind de la forma inițială prin mutarea locului liber a.î. să ajungem la forma finală

- $f(n) = g(n) + h(n)$

  $g(n) = g(n') + c(n', n)$

  are nevoie de euristică ajutătoare pentru a-l ghida.

- OPEN = mulțimea nodurilor explorate (frontiera dintre zona cunoscută și cea necunoscută)
- CLOSED = mulțimea nodurilor explorate (regiunea cunoscută în totalitate)
- complexitate:
  - liniară: $|h^*(n) - h(n)| \leq \delta$, δ const
  - subexponențială: $|h^*(n) - h(n)| \leq O(\log(h^*(n)))$
  - exponențială

9. Complexitate Edmons-Karp

temporală: $O(n \cdot m^2)$ / $O(|V| * |E|^2)$

(creștem fluxul pe un drum de ameliorare, o muchie de drum saturată, dacă vreodată această muchie va fi folosită în alt drum de ameliorare, aceasta va fi folosită într-un drum mai lung, respectiv de distanțe sunt monotone și limitate de $O(|V|)$

- spațială: $O(n)$, $O(|V|)$ - coadă pt BFS + vectori de părinți pentru a putea parcurge drum de ameliorare

Scanned with CamScanner

- 2 euristici pentru problema aranjării a 8 pătrate

h_1 = nr. pătrate a căror poziție diferă de cea finală

h_2 = suma distanțelor Manhattan între poziția finală și curentă, pt fiecare dintre cele 8 pătrate

- euristică admisibilă la A*: $0 \leq h(u) \leq h^*(u)$, dacă nu ar fi admisibilă, A^* ar găsi o soluție, dar nu se garantează că e optim
- dominanță: Un alg A_1^* domină un alg A_2^*, dacă orice nod expandat de A_1^* e expandat și de A_2^*
- demonstrație că orice euristică consistentă e admisibilă:

Pentru ca o euristică să fie admisibilă, trebuie să nu supraestimeze costul atingerii scopului. Dacă se întâmplă asta, A* poate ignora drumul cel mai scurt deoarece valoarea supraestimată a drumului inițial pare mai mare decât altă posibilă

O euristică consistentă estimează mereu distanța de la un nod vecin la destinație ca fiind mai mică sau egală cu costul real. În acest caz, euristica e admisibilă.

monotonă: $h(n) \leq h(n') + c(n,n')$ ← costul arcului

- Algoritmul A*
  - găsirea de drumuri și traversarea grafurilor, căutând o cale directă între noduri

Scanned with CamScanner

10 Algoritmul Floyd-Warshall

- Se poate folosi matricea finală de distanțe dacă graful are cicluri negative?
  - pe diagonală pot apărea valori diferite de 0, deoarece dacă există un nod ce aparține unui ciclu, la final va ajunge în același nod cu cost <0
  - acest algoritm poate determina cicluri de cost negativ
- Modificare pentru închidere tranzitivă
  - folosind o matrice cu valori de tip boolean (dacă există muchie de la i la j valoarea e 1, altfel 0)
  - în loc de operatori ~~logici~~ aritmetici, folosim operatori logici (&&, ||)
- Rezultatele sunt corecte doar în cazul unor cicluri negative

11 Divide et impera

Pași
- Divide: împarte problema inițială într-un număr mai mic de subprobleme mai mici de aceeași natură
- Impera: rezolvă subproblemele recursiv, dacă dimensiunea subproblemei e mică se rezolvă iterativ
- Combină: soluțiile subproblemelor pentru a obține soluția problemei inițiale

$T(n) = D(n) + S(n) + C(n)$

ex: Binary Search, MergeSort

Scanned with CamScanner

12. Arbori Huffman

"arbore cu muchii"

c -> cod[c]

cost[fraza] = ?, cost muchie { a 3 e 3 m 1 m 1 n 2 _ 2

cost fraza: $\sum_{c \in V}$ frecv(c) * lungime_cod[c]

frecventa de aparitie (raportat la nr de caract)

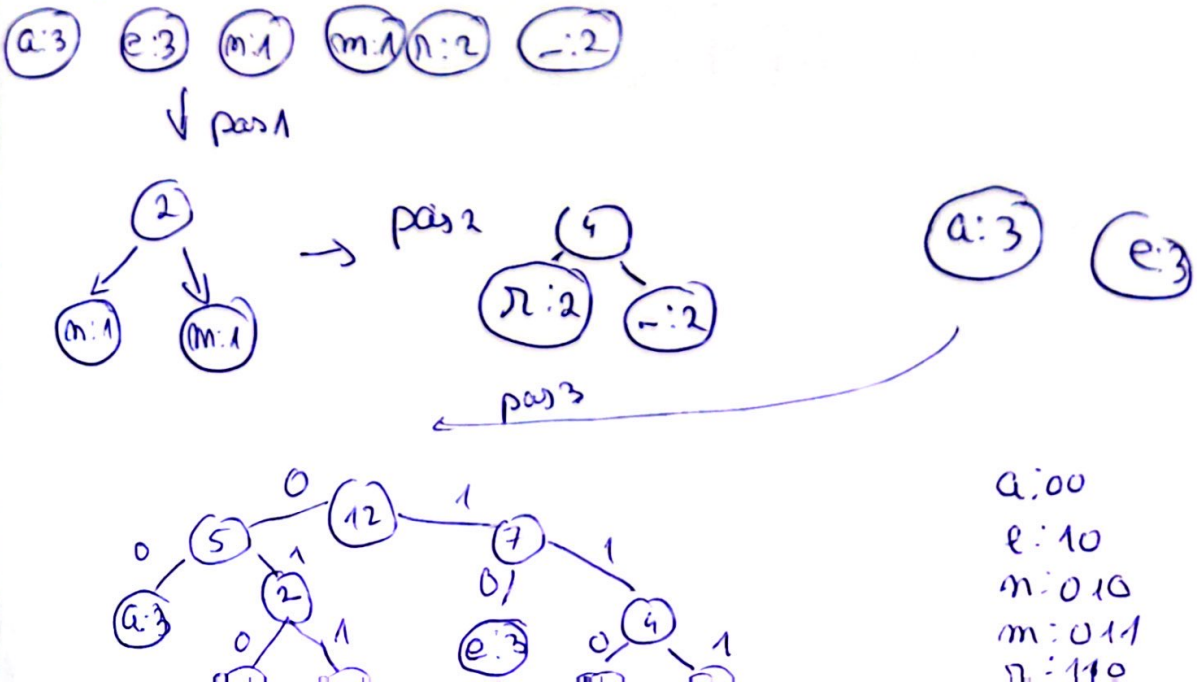
- sortam dupa frecv, adica de cate ori apare in text (nu neaparat la nr de caractere)

- codificare = merg in jos pana la frunze, daca merg pe subarborele stang adaug 0, altfel 1. In final, codificarea frunzei este secventa rezultata.

- dupa ce am obtinut codificarea pentru fiecare caracter, o inlocuiesc in sir pentru a obtine codificarea sirului

- decodificare: stringul de biti in cati pasi ma intorc

- algoritm



Scanned with CamScanner

13 Arbori minimi de acoperire

- Algoritmul Prim:
  - strategie constructivă greedy
  - merge pe muchiile de cost minimum, din aproape în aproape, considerând o sursă inițială arborii aleatori, extinzând MST-ul curent cu muchia de cost minim neadaugată încă la arbore.
  - complexitate: depinde asem cu Dijkstra:
    $O(V^2)$ - matrice de adiacență
    $O(E \cdot \log V)$ - Heap binar
    $O(V \log V + E)$ - Heap Fibonacci
  - corectitudine:
    1. arătăm că muchiile pe care le adăugăm aparțin Arb prin inducție după muchiile adăugate în AMA
    2. arătăm că muchiile ignorate nu fac parte din Arb

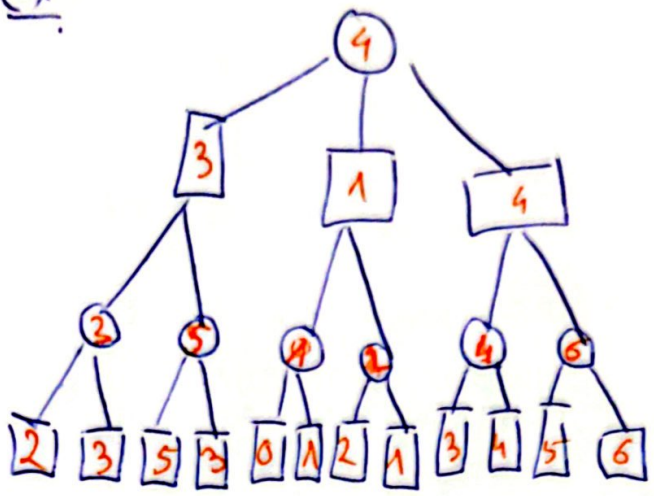
- Algoritm ~~Kruskal~~

- rezolvă MST-problem în grafuri neorientate cu costurile muchiilor oarecare, aplicând o strategie greedy foarte simplă: muchiile de cost minim probabil fac parte din MST
- complexitate: depinde de implementarea mulțimilor disjuncte
- corectitudine:
  1. arătăm că muchiile ignorate nu fac parte din Arb
  2. arătăm că muchiile pe care le adăugăm ↑
     [illegible] inducție după muchiile adăugate în AMA

Scanned with CamScanner

MAX

Ex:



MAX
MIN
MAX
MIN

- Joc de șah: - construim arborele stărilor până la o adâncime anume
  - alegem mutările examinate cu ajutorul euristicilor.
  - Ex: • mutări în care se iau piese
    • mai întâi mutări cu scor mai bun
    • mutări care au mai generat tăieri

- modificarea condiției de tăiere din $\beta < \alpha$ în $\beta - k < \alpha$, k constantă

1. $\beta = k$ ⇒ alg nu se schimbă
2. $\beta - k < \beta$ ⇒ riscăm să facem tăieturi inutile
3. $\beta - k > \beta$ ⇒ Riscăm să omitem tăieturi ce va duce la creșterea timpului de rulare al alg.

- Tăieturi α-β
  - optimizări minimax
  - pe parcursul ~~eliminării~~ examinării arborelui de soluții se pot elimina întregi subarbori corespunzători unei mișcări mi, dacă pe parcursul analizei găsim că mișcarea mi e mai slabă decât mișcarea curentă.
  - α = max dintre găsite pentru un nod MAX

Scanned with CamScanner

- tăiem o ramură dacă:
  - am găsit un nod pe nivel Max cu val $\beta <=$ $\forall \alpha$ calculate anterior
  - am găsit un nod pe nivelul Min cu val $\alpha >=$ $\forall \beta$ calculate anterior
- reduce complexitatea minimax

- ambii algoritmi: ⟨ complexitate foarte mare
  soluții euristice pentru limitarea complexității

- optimizări:
  - limitarea adâncimii căutării
    - trebuie să construim o funcție euristică care să estimeze șansele de câștig pentru poziția dată
  - oprirea căutării (după un nr maxim de nivele, când profitul obținut din continuarea căutării devine foarte mic.

15. Grafuri

- BFS - coadă
  - reținem părintele, distanța până la nodul sursă, culoarea nodului
  - complexitate $O(n+m)$
- DFS - timpi de start și de finalizare, părinte, culoare
  - stivă
  - complexitate $O(n+m)$

Scanned with CamScanner

Tipuri de arce în DFS

- arborele de parcurgere DFS - toate nodurile din graf împreună cu muchiile vizitate de DFS.

* muchia de arbore (T) = muchie (x, y) conectează un nod x de copilul său y în arbore

* ~~back~~ muchie înapoi (B) = muchie (x, y) care conectează un nod x de un strămoș y.

- graf orientat:

* muchie înainte (F) = muchie (x, y) care nu e T și care conectează un nod x de un descendent y

* ~~cross~~ muchie de traversare (C) = muchie (x, y) care conectează un nod x de un nod y din alt subarbore.

Algoritmul Tarjan

- explicații: found[node] - timpul de start din DFS.
low_link[node] - cel mai mic timp de descoperire la care se poate ajunge pornind din node și mergând pe arcele/muchiile nevizitate

Scanned with CamScanner

Complexitate - temporală $O(n+m)$
- spațială $O(n)$

Algoritm Kosaraju

- compactare cicluri
- DFS pe graful transpus
- aceeași complexitate ca Tarjan

Puncte de articulație

- nod în graf neorientat a cărui eliminare duce la creșterea nr. de comp. conexe
- se elimină nodul cu muchiile incidente

Punți critice

- muchie în graf neorientate a cărui eliminare duce la creșterea nr. de comp. conexe
- se elimină muchia, fără a șterge nodurile

Componente Biconexe

- un graf neorientat, submulțime maximală de noduri cu prop. că nu conține puncte de articulație

- un graf neorientat e biconex dacă nu conține puncte de articulație - conține o singură.

Scanned with CamScanner